WIKINOTES · OFFLINE EDITION

Terraform Associate 004

A complete study guide for your flight.

Exam target: Terraform 1.12

All 18 source documents are included. Start with the final cram sheet, then select a topic from Contents.

Read without internet

The chapters, examples, answers, and links between guides are stored inside this book.

Links to Apple, HashiCorp, and other websites need internet access. Those external pages are not included.

Read on a small screen

This PDF uses small pages for portrait reading. Tables appear as stacked entries. Long code lines wrap on screen.

Wrapping changes the display, not the code. Landscape orientation gives examples more space.

The original Markdown files remain the editable source. This edition is a snapshot.

Contents

1. Terraform Associate 004: Final Cram Sheet
2. Terraform Associate 004: Study Map
3. Terraform Infrastructure as Code
4. Terraform Providers and Versions
5. Terraform Core Workflow (Associate 004)
6. Terraform Block Types (Associate 004)
7. Terraform Variables, Types, and Outputs
8. Terraform Variable Precedence (Associate 004)
9. Terraform Expressions and Functions
10. Terraform Dependencies and Lifecycle
11. Terraform Custom Conditions
12. Terraform Sensitive Data and Ephemeral Values
13. Terraform Modules
14. Terraform State and Backends
15. Terraform State Operations and Imports
16. Terraform Debugging

17. HCP Terraform

18. Terraform Tags on AWS

CHAPTER 01 / 18

Terraform Associate 004: Final Cram Sheet

Target: Terraform 1.12. **Reviewed:** 2026-10-14. Use this for final recall. Open the study map for examples.

1. Remember the model

Configuration: desired result. **Provider:** API plugin. **State:** recorded object bindings. **Plan:** proposed actions. **Apply:** execute actions and save state. IaC supports repeatability and review. Multi-cloud workflows still need provider-specific resource types.

2. Select the command

Need / Command

Install dependencies and initialize storage

```
terraform init
```

Select newer compatible dependencies

```
terraform init -upgrade
```

Check formatting without edits

```
terraform fmt -check
```

Check configuration consistency

```
terraform validate
```

Save proposed changes

```
terraform plan -out=tfplan
```

Apply that exact plan

```
terraform apply tfplan
```

Destroy managed resources

`terraform destroy`

`fmt` needs no initialization. `validate` needs installed dependencies. Validation does not prove cloud permissions. Plain `apply` replans. Saved-plan apply does not ask for approval again.

```
+ create    ~ update    - destroy
-/+ destroy then create    +/- create
then destroy
```

3. Separate version controls

Setting / Meaning

`required_version`

CLI constraint

`required_providers`

Provider sources and constraints

`provider` block

Connection settings

`.terraform.lock.hcl`

Selected provider versions and checksums

Module `version`

Registry module constraint

`~> 5.20` allows later 5.x versions. `~> 5.20.0` allows only later 5.20.x versions. The lock file covers providers, not modules or state writes. Commit it. Aliases select provider configurations, not different versions.

4. Predict the value and address

Local CLI precedence, weakest first:

```
default < TF_VAR_* < terraform.tfvars
< terraform.tfvars.json
```

```
< *.auto.tfvars /
*.auto.tfvars.json < -var / -var-file
```

Auto files load alphabetically. CLI assignments load in supplied order. Last wins. `prod.tfvars` needs `-var-file`. HCP has extra precedence rules.

Construct / Remember

`resource` / `data`

Manage an object / read information

List / set

Ordered with duplicates / unique members without numeric indices

Map / object

One element type / named attributes with specified types

`null` / unknown

No value / value not yet known

`count`

Numeric addresses: `thing.example[0]`

`for_each`

Keyed addresses: `thing.example["api"]`

`for` expression

Transform values

`dynamic`

Generate nested blocks

Resource `for_each` needs a map or set of strings. Keys must be known and non-sensitive. `merge(a, b)` gives conflicting keys to `b`. `try` handles dynamic errors. `can` tests successful evaluation.

5. Dependencies and conditions

References set dependencies. File order does not. Use `depends_on` for hidden dependencies.

Setting / Remember

`create_before_destroy`

Create replacement first. Naming and capacity limits still apply.

`prevent_destroy`

Reject destruction while the rule remains in configuration

`ignore_changes`

Ignore selected attributes during update planning

`validation`

Reject invalid inputs

`precondition`

Reject an invalid assumption

`postcondition`

Reject an invalid result. Block dependent work.

`check`

Warn on a false assertion

Unknown values can defer conditions. Outputs allow preconditions, not postconditions.

6. Modules and state operations

Pass child inputs explicitly. Read `module.name.output`. Call child directories with `module` blocks. Map provider aliases explicitly. Use `version` for registry modules and `?ref=` for Git sources.

Need / Use

Inspect addresses

`terraform state list`

Inspect one object

`terraform state show ADDRESS`

Rename without replacing

`moved` block

Stop management, retain object

`removed` with `destroy = false`

Adopt an existing object

`import` block or `terraform import`

Update state without changing infrastructure

Refresh-only plan/apply

Copy state to another backend

`terraform init -migrate-state`

CLI import does not generate configuration. Refresh-only does not edit it. Normal apply can reverse drift. Remote state does not imply remote execution. Force-unlock only your confirmed stale lock. Locking support depends on the backend.

7. Secrets and logs

Sensitive hides. Ephemeral omits.

- Sensitive values can remain in state and plans.
- Ephemeral values need permitted contexts. Root outputs cannot be ephemeral.
- Write-only arguments are not retained.
- Normal Vault data sources can store secrets in state.
- `TF_LOG=TRACE`: maximum detail. `TF_LOG_PATH`: log file. A path alone enables nothing.

8. HCP Terraform

Term / Remember

Workspace

Configuration context, state, variables, and runs

Project

Group workspaces and access

CLI workspace

Separate state for one working configuration

CLI/VCS/API workflow

How a run starts

Remote/agent/local execution

Where Terraform runs

`cloud` block

HCP integration. Cannot coexist with `backend`.

`terraform login`

HCP authentication, not cloud-provider authentication

Variable set

Shared inputs. Priority sets can override narrower scopes.

Run trigger

Queue a downstream run after a successful upstream apply

Sentinel / OPA

Policy enforcement

Health assessment

Detect drift and failed checks

A trigger does not pass outputs. A speculative plan cannot be applied. Also recognize teams, private registry, run tasks, Explorer, change requests, and dynamic credentials.

Coverage follows the 004 objectives. Use the topic guides for references and self-tests.

Terraform Associate 004: Study Map

Exam target: Terraform 1.12. **Reviewed:** 2026-10-14. HashiCorp defines eight exam sections. Cloud-provider-specific knowledge is not required. See the official objectives.

Start here

1. Read the final cram sheet.
2. Identify a topic you cannot explain.
3. Open its guide below.
4. Read the examples.
5. Answer the self-test before you read the answers.

The guides use short sentences, direct instructions, and consistent terms. This follows the intent of ASD-STE100. Terraform names remain unchanged. This is not a certified STE translation.

Find each exam objective

Objectives / Know how to... / Read

1a–1c

Know how to...: Explain IaC, its benefits, and multi-cloud use

Read: IaC

2a–2c

Know how to...: Install, select, configure, and combine providers

Read: Providers and versions

2d

Know how to...: Explain the purpose of state

Read: State and backends

3a–3g

Know how to...: Write, initialize, validate, plan, apply, destroy, and format

Read: Core workflow

4a

Know how to...: Select a resource or data source

Read: Block types

4b, 4f

Know how to...: Reference resources and control dependencies

Read: Dependencies and lifecycle

4c–4d

Know how to...: Use variables, outputs, and complex types

Read: Variables and types

4c

Know how to...: Select the winning input value

Read: Variable precedence

4e

Know how to...: Use expressions and functions

Read: Expressions

4g

Know how to...: Select and use custom conditions

Read: Custom conditions

4h

Know how to...: Protect secrets, including Vault use

Read: Sensitive data

5a–5d

Know how to...: Source, call, and version modules

Read: Modules

6a–6c

Know how to...: Configure storage and explain locking

Read: State and backends

6d, 7a–7b

Know how to...: Handle drift, import resources, and inspect state

Read: State operations

7c

Know how to...: Enable and use verbose logs

Read: Debugging

8a–8d

Know how to...: Use HCP Terraform, projects, and collaboration features

Read: HCP Terraform

The official learning path provides further exercises for these objectives.

Use the examples correctly

Label / Meaning

Local lab

A complete example. Use a separate, empty directory.

Excerpt

Part of a configuration. Other settings or resources are required.

Console

An expression to enter in `terraform console`.

Expected

The result to compare with your result.

The local labs use the built-in `terraform_data` resource where practical. They do not require a cloud account. Each lab has its own state.

Cloud examples explain Terraform behavior. Replace example names before real use. An `apply` command can create billable resources when you use a cloud provider.

Terraform CLI versions and provider versions are separate. Provider version constraints in examples are teaching examples, not upgrade recommendations.

Check your readiness

Exercise / Expected skill	
Run the core-workflow lab	Explain each command and plan symbol
Run the modules lab	Pass a root input to a child and return an output
Change a `for_each` key	Predict the resource addresses and replacement effects

Rename a resource with `moved`

Keep its existing state binding

Compare normal and refresh-only plans

Distinguish infrastructure changes from state updates

Explain an HCP workspace

Identify its state, variables, runs, and access controls

Explain `sensitive` and `ephemeral`

Identify what can remain in state

Use the official sample questions to learn the question formats. They are not a complete practice exam.

The AWS tags guide is optional practical material. Study the eight exam sections first.

CHAPTER 03 / 18

Terraform Infrastructure as Code

Associate 004 · Terraform 1.12 · Objectives 1a–1c | Study map

The main idea

Infrastructure as Code (IaC) describes infrastructure in files. Terraform compares the desired configuration with existing objects. It proposes actions that bring those objects into the desired state.

Term / Meaning / Example

> **Configuration**
>
> **Meaning:** Your description of the desired result
>
> **Example:** Two application instances

Provider

Meaning: A plugin that communicates with an API

Example: AWS, Azure, GitHub

Resource

Meaning: An object Terraform manages

Example: A network or repository

State

Meaning: Terraform's record of managed objects

Example: A resource address mapped to an object ID

Plan

Meaning: Proposed actions

Example: Add one instance

Apply

Meaning: Execute the proposed actions

Example: Create that instance

Reference: What is Terraform?

Declarative and imperative instructions

Approach / Example / Who selects the operations?

> **Imperative**
>
> **Example:** Create a network. Then create a server.
>
> **Who selects the operations?:** The script author

> **Declarative**
>
> **Example:** A server must exist in this network.
>
> **Who selects the operations?:** Terraform and its providers

Excerpt: Declare an AWS network and a subnet.

```
resource "aws_vpc" "lab" {
  cidr_block = "10.20.0.0/16"
}
```

```
resource "aws_subnet" "app" {
  vpc_id     = aws_vpc.lab.id
  cidr_block = "10.20.1.0/24"
}
```

The reference creates a dependency. Terraform creates the network before the subnet. The order of the blocks in the file does not set the execution order.

Why use IaC?

Need / IaC benefit / Example

> **Repeat an environment**
>
> **IaC benefit:** Reuse configuration
>
> **Example:** Create a second test environment

> **Review a change**
>
> **IaC benefit:** Use version control and plans
>
> **Example:** Review a proposed subnet change

Find the cause of a change

IaC benefit: Keep configuration history

Example: Find the commit that changed an instance size

Reduce manual differences

IaC benefit: Use one declared configuration

Example: Apply the same tags to each environment

Recover infrastructure

IaC benefit: Reuse configuration and protected state

Example: Rebuild infrastructure after a failure

Work as a team

IaC benefit: Share code and remote state

Example: Review code before another person applies it

IaC does not back up application data. A database still needs its own backup process. Reusing code does not copy the state of an existing environment.

Multi-cloud does not mean identical resources

```
One Terraform workflow
  ├── AWS provider       → AWS network
  ├── Azure provider     → Azure resource group
  └── GitHub provider    → GitHub repository
```

The workflow is consistent. The resource schemas remain provider-specific. An `aws_instance` cannot create an Azure virtual machine.

Deployment / Example

> **Multi-cloud**
>
> AWS application and Azure recovery infrastructure

Hybrid cloud

On-premises virtual machines and a public-cloud network

Service-agnostic workflow

Cloud infrastructure plus DNS and GitHub configuration

Terraform needs a suitable provider and API access for each system. Reference: Terraform use cases.

Predict the next plan

```
Configuration: two instances
Real objects:  one managed instance
Plan:          create the missing
instance

Configuration: two instances
Real objects:  two matching managed
instances
Plan:          no changes
```

Stable configuration should produce no changes when the real objects already match. External changes, changing input data, or provider behavior can produce a new plan.

Self-test

1. Does Terraform use file order to select resource order?
2. Can one AWS resource block create an Azure resource?
3. Does version-controlled configuration replace a database backup?
4. Which artifact lets a reviewer inspect proposed infrastructure changes?

Answers: 1. No. It uses dependencies. 2. No. 3. No. 4. The plan.

Terraform Providers and Versions

Associate 004 · Terraform 1.12 · Objectives 2a–2c, 3b | Study map

Separate the three versions

Setting / Controls

> **`required_version`**
> Terraform CLI version

> **`required_providers.aws.version`**
> AWS provider version

> **`module.version`**
> A registry module version

Excerpt: Require two plugins. Configure the AWS connection separately.

```
terraform {
  required_version = "≥ 1.12.0, <
2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "⇝ 5.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "⇝ 3.7"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "random_pet" "name" {
  length = 2
}

resource "aws_s3_bucket" "lab" {
  bucket_prefix =
"exam-${random_pet.name.id}-"
}
```

`random` does not need a configuration block for this example. The AWS provider uses its supported credential sources. Do not place credentials in the code.

Reference: Provider requirements.

Read a source address

```
registry.terraform.io / hashicorp /
aws
hostname                namespace
type
```

`hashicorp/aws` uses the default hostname, `registry.terraform.io`. The local name `aws` connects resource types and provider configurations to that requirement. Providers define resource schemas, data sources, and API operations. Terraform Core evaluates configuration and builds the execution graph.

Read version constraints

Constraint / Accepted versions

`= 5.20.0`

Exactly 5.20.0

`⩾ 5.20.0`

5.20.0 and later

`⩾ 5.20.0, < 6.0.0`

5.20.0 through the remaining 5.x versions

`⇝ 5.20`

At least 5.20.0, but below 6.0.0

`⇝ 5.20.0`

At least 5.20.0, but below 5.21.0

`≠ 5.20.1`

Exclude 5.20.1

All applicable constraints must agree. Terraform selects one version of each provider source for the configuration. Two AWS aliases do not install two AWS provider versions.

Reference: Version constraints.

Initialization and the dependency lock file

```
terraform init
terraform providers
terraform init -upgrade
```

Situation / Result

> **No selection exists in the lock file**
>
> `init` selects a compatible provider version

> **A compatible selection exists**
>
> `init` reuses the locked version

> **You run `init -upgrade`**
>
> Terraform can select newer versions within the constraints

> **Constraints exclude the locked version**
>
> Update the selection with `init -upgrade`

Commit `.terraform.lock.hcl`. It records selected provider versions and package checksums. Do not commit `.terraform/`. It contains local initialization data and downloaded dependencies. The dependency lock file does not lock remote module versions. It does not prevent concurrent state writes.

Reference: Dependency lock file.

Use two configurations of one provider

Excerpt: Add an alias for the second region.

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "west"
  region = "us-west-2"
}

resource "aws_s3_bucket" "archive" {
  provider      = aws.west
  bucket_prefix = "exam-archive-"
}
```

`provider = aws.west` selects a provider configuration. It is not a string. Resources without this argument use the default configuration. If every configuration has an alias, Terraform creates an implied empty default configuration. That default can fail when required settings are missing.

Pass aliases into child modules explicitly. See the modules guide. Reference: Provider block.

Self-test

1. Does `~> 5.20.0` allow 5.21.0?
2. Does `init -upgrade` ignore the configured version constraints?
3. Does `.terraform.lock.hcl` lock module versions?
4. Which block configures a region: `required_providers` or `provider`?

Answers: 1. No. 2. No. 3. No. 4. `provider`.

CHAPTER 05 / 18

Terraform Core Workflow (Associate 004)

Associate 004 · Terraform 1.12 · Objectives 3a–3g | Study map

Use this sequence to create, inspect, change, and remove managed infrastructure.

The order

```
1. Write    →  create .tf files
2. Providers →  declare
required_providers + provider blocks
3. init     →  terraform init
4. fmt      →  terraform fmt
5. validate →  terraform validate
6. plan     →  terraform plan
7. apply    →  terraform apply
8. destroy  →  terraform destroy
```

Mnemonic: Why **P**eople **I**n **F**ield **V**isits **P**lan **A**head **D**aily (Write, Providers, Init, Fmt, Validate, Plan, Apply, Destroy)

HashiCorp's official short form is **Write → Plan → Apply**; everything else fits inside or around those three.

Each phase

/ Phase / What it does / Remember

1

Phase: **Write**

What it does: Create `.tf` files (`main.tf`, `variables.tf`, `outputs.tf`) in HCL.

Remember: Terraform loads every `.tf` file in the directory; file names don't matter to it.

2

Phase: **Providers**

What it does: `required_providers` declares sources and version constraints. A `provider` block configures a connection.

Remember: Providers are plugins that communicate with APIs.

3

Phase: **init**

What it does: Downloads providers and modules, sets up the backend, writes `.terraform/` and `.terraform.lock.hcl`.

Remember: **Run first, and again** whenever you add/change a provider, module, or backend. Safe to re-run.

4

Phase: **fmt**

What it does: Rewrites files to canonical HCL style (indentation, alignment).

Remember: Style only — no logic check. `-check` fails without changing files (CI); `-recursive` for subdirs.

5

Phase: **validate**

What it does: Checks syntax and internal consistency (types, required args, references).

Remember: Needs `init` first (needs provider schemas). Does **not** contact real infrastructure.

6

Phase: **plan**

What it does: Normally reads real objects and compares them with configuration and recorded state.

Remember: Does not apply infrastructure changes. `-out=tfplan` saves the plan.

7

Phase: **apply**

What it does: Executes changes and writes state to the configured backend.

Remember: Without a saved plan, it creates a new plan and normally asks for approval.

8

Phase: **destroy**

What it does: Plans and applies removal of managed resources in the selected state.

Remember: Same as `terraform apply -destroy`. Data sources do not cause object deletion.

Reference: Core workflow.

Command-order details

- **`fmt` does not require `init`.** It only reads text, so it can run any time. `validate`

does require `init`. Order `init → fmt → validate` is the conventional sequence.

- **`plan` is optional before `apply`.** Plain `apply` creates its own plan. Review the plan before approval. Use a saved plan for an automated apply.

Complete local lab

Save this as `main.tf` in an empty directory. It uses no external provider or cloud account.

```
variable "message" {
  type    = string
  default = "first version"
}

resource "terraform_data" "example" {
  input = var.message
}

output "message" {
  value =
terraform_data.example.output
}
```

```
terraform init
terraform fmt -check
```

```
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
terraform output -raw message
terraform plan
```

Expected: Create one resource. Output `first version`. The final plan proposes no changes. Applying `tfplan` executes the saved plan without another approval prompt. Treat saved plans as sensitive files. They can contain secret input values.

Change the input and review the proposed update:

```
terraform plan -var='message=second
version' -out=update.tfplan
terraform apply update.tfplan
terraform plan -destroy
terraform destroy
```

The final command asks for confirmation before it removes the lab resource.

Read plan symbols and options

Symbol or option / Meaning

+

Create

~

Update in place

-

Destroy

-/+

Destroy the old object, then create its replacement

+/-

Create the replacement, then destroy the old object

≤

Read a data source

(known after apply)

The final value is not yet available

-replace=ADDRESS

Request replacement of a managed resource

-refresh-only

Plan state and output updates without infrastructure changes

-detailed-exitcode

Return 0 for no changes, 1 for an error, or 2 for changes

Saved-plan apply uses the plan's recorded decisions and input values. Do not supply new normal `-var` values when applying that file. Ephemeral inputs are not saved and may need to be supplied again.

`validate` does not check cloud permissions, quotas, or real-object existence. During apply, API errors can leave a partially

completed deployment. Correct the cause and make a new plan. Terraform does not automatically roll back completed work.

References: Plan and apply.

Quick self-test

1. Which command creates `.terraform.lock.hcl`? → `init`
2. Which command fails if you never ran `init`? → `validate` (and `plan`/`apply`)
3. Which command changes no infrastructure but contacts providers to read real state? → `plan`
4. You added a new module. What must you run before `plan`? → `init`
5. Which command is used in CI to fail on unformatted code? → `terraform fmt -check`
6. What's the equivalent of `terraform destroy`? → `terraform apply -destroy`

Terraform Block Types (Associate 004)

Target: Terraform 1.12. | Study map
Examples are excerpts unless stated otherwise. Configure required providers and omitted inputs before use.

Every Terraform file is built from **blocks**: `type "label" "label" { ... }`. This sheet lists each top-level block, then the blocks that only live inside other blocks.

At a glance

Group / Block / One-line purpose

> **Settings**
>
> **Block:** `terraform`
>
> **One-line purpose:** Terraform version, providers, and where state lives

Settings

Block: `provider`

One-line purpose: Configure a plugin (region, credentials, default tags)

Infrastructure

Block: `resource`

One-line purpose: Create and manage something

Infrastructure

Block: `data`

One-line purpose: Read something that already exists

Infrastructure

Block: `ephemeral`

One-line purpose: Obtain or generate temporary values that Terraform omits from state and plans

Infrastructure

Block: `module`

One-line purpose: Reuse a group of blocks as one unit

Inputs / values / outputs

Block: `variable`

One-line purpose: Input from outside the configuration

Inputs / values / outputs

Block: `locals`

One-line purpose: Named internal values

Inputs / values / outputs

Block: `output`

One-line purpose: Expose values to the CLI or a parent module

State refactoring

Block: `moved`

One-line purpose: Rename or move a resource without destroying it

State refactoring

Block: `import`

One-line purpose: Bring an existing resource under Terraform

State refactoring

Block: `removed`

One-line purpose: Stop managing a resource, optionally without destroying it

Validation

Block: `check`

One-line purpose: Assertions evaluated during operations. False assertions warn.

Memory aid: a configuration has **settings** (how Terraform runs), **things** (what it builds or reads), **values** (what flows in and out), **refactoring** (how state changes safely), and **checks**.

Settings

`terraform`

```
terraform {
  required_version = "≥ 1.12.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "↝ 5.0"
    }
  }

  backend "s3" {
    bucket       = "my-tf-state"
    key          =
"lab/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
```

```
  }
}
```

Best used: declare CLI/provider constraints and optional remote-state settings. Child modules also declare their provider requirements. These settings do not accept ordinary variable or resource references.

`provider`

```
provider "aws" {
  region = var.region
}

provider "aws" {
  alias  = "west"
  region = "us-west-2"
}
```

Best used: to configure how a plugin connects. Use `alias` when you need the same provider twice, such as two regions or two accounts. A resource selects it with `provider = aws.west`.

Infrastructure

`resource`

```
resource "aws_s3_bucket" "logs" {
  bucket = "wikinotes-lab-logs"
}
```

Best used: for anything Terraform should **create, update, and destroy**. Reference it as `<type>.<name>.<attribute>`, for example `aws_s3_bucket.logs.arn`.

`data`

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-
gp3/ubuntu-noble-24.04-amd64-server-
*"]
  }
}

resource "aws_instance" "web" {
  ami           =
```

```
data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
}
```

Best used: read information, such as an AMI, an existing VPC, or the current account ID. A data source can also read an object managed elsewhere in the configuration. It does not manage that object's lifecycle. Reference it with a `data.` prefix.

`ephemeral` (Terraform 1.10+)

The write-only database argument below also requires Terraform 1.11+ and a supporting provider version.

```
ephemeral
"aws_secretsmanager_secret_version"
"db" {
  secret_id = "prod/db-password"
}

resource "aws_db_instance" "main" {
  # ...
  password_wo         =
ephemeral.aws_secretsmanager_secret_v
ersion.db.secret_string
```

```
  password_wo_version = 1
}
```

Best used: for **secrets** you need during a run but never want written to the plan or state file. Ephemeral values can only go into places that are also ephemeral, such as provider settings or write-only (`_wo`) arguments.

`module`

```
module "vpc" {
  source  = "terraform-aws-
modules/vpc/aws"
  version = "~> 5.0"

  name = "lab-vpc"
  cidr = "10.0.0.0/16"
}
```

Best used: when the same group of resources is needed more than once (per environment, per team) or to use a tested public module. Run `terraform init` after adding or changing one. Reference outputs with `module.vpc.vpc_id`.

Inputs, values, and outputs

`variable`

```
variable "environment" {
  type        = string
  description = "Deployment
environment"
  default     = "dev"

  validation {
    condition     = contains(["dev",
"staging", "prod"], var.environment)
    error_message = "environment must
be dev, staging, or prod."
  }
}
```

Best used: for anything that **changes between deployments**: region, environment, instance size. Reference with `var.environment`. Add `sensitive = true` to hide a value from CLI output (it is still stored in state).

locals

```
locals {
  name_prefix =
"${var.project}-${var.environment}"
  common_tags = { Project =
var.project, Environment =
var.environment }
}
```

Best used: to **name a value you'd otherwise repeat** or to compute one from other values. Callers can't set locals. Note that the block is `locals` (plural) but references use `local.name_prefix` (singular).

output

```
output "bucket_arn" {
  value       =
aws_s3_bucket.logs.arn
  description = "ARN of the logs
bucket"
}
```

Best used: to **print a value after `apply`**, to pass data from a child module to its parent, or to share values with another configuration (through `terraform_remote_state`). Read them with `terraform output`.

State refactoring

`moved` (Terraform 1.1+)

```
moved {
  from = aws_instance.web
  to   = aws_instance.frontend
}
```

Best used: when you **rename a resource or move it into a module**. Without a state move, Terraform normally proposes destroying the old address and creating the new one. This is a declarative alternative to `terraform state mv`.

`import` (Terraform 1.5+)

```
import {
  to = aws_s3_bucket.legacy
  id = "legacy-bucket-name"
}
```

Best used: when a resource was **created by hand** (console, CLI) and you want Terraform to manage it. `terraform plan -generate-config-out=generated.tf` writes a starting `resource` block when the target configuration is missing. This is a declarative alternative to `terraform import`.

`removed` (Terraform 1.7+)

```
removed {
  from = aws_s3_bucket.legacy

  lifecycle {
    destroy = false
  }
}
```

Best used: to **stop managing a resource while leaving it running** in AWS, for example when another team takes it over. With `destroy = false`, it is dropped from state only. This is a declarative alternative to `terraform state rm`.

Validation

check (Terraform 1.5+)

```
check "website_up" {
  data "http" "site" {
    url = "https://example.com"
  }

  assert {
    condition     =
data.http.site.status_code == 200
    error_message = "Website did not
return 200."
  }
}
```

Best used: for **health checks that shouldn't block a deployment**. A failed `check` prints a **warning**; `plan` and `apply` still continue. To block a run instead, use `precondition`/`postcondition` (below).

Nested blocks

These only appear inside another block.

Block / Lives inside / Best used for

> **`required_providers`**
>
> **Lives inside:** `terraform`
>
> **Best used for:** Declaring provider sources and version constraints

> **`backend`**
>
> **Lives inside:** `terraform`
>
> **Best used for:** Storing state remotely (S3, azurerm, gcs, ...)

`cloud`

Lives inside: `terraform`

Best used for: Connecting to HCP Terraform instead of a `backend`

`default_tags`

Lives inside: `provider "aws"`

Best used for: Tags on every resource from that provider

`validation`

Lives inside: `variable`

Best used for: Rejecting bad input when required values become available

`lifecycle`

Lives inside: `resource`, `data`, `ephemeral`, `removed`

Best used for: Behavior depends on the parent. Not every parent accepts every rule.

`precondition`

Lives inside: `lifecycle`, or directly in `output`

Best used for: Check assumptions. False conditions produce errors.

`postcondition`

Lives inside: Resource/data/ephemeral `lifecycle`

Best used for: Check results. False conditions block dependent work.

`dynamic`

Lives inside: any block that accepts repeated nested blocks

Best used for: Generating repeated nested blocks from a list or map

`provisioner`

Lives inside: `resource`

Best used for: Running a script on create/destroy. Last resort; prefer `user_data` or config tools

`connection`

Lives inside: `resource`, `provisioner`

Best used for: SSH/WinRM details for a `remote-exec` or `file` provisioner

`lifecycle` options

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
# build the new one first, then
delete the old one
    prevent_destroy       = true
# error if a plan would destroy this
    ignore_changes        =
[tags["LastScan"]]   # don't undo
changes made outside Terraform
    replace_triggered_by  =
[aws_launch_template.web.latest_versi
on]
```

```
    precondition {
      condition     = var.environment
≠ "prod" || var.instance_type ≠
"t3.micro"
      error_message = "prod needs
something bigger than t3.micro."
    }
  }
}
```

dynamic

```
resource "aws_security_group" "web" {
  name = "web"

  dynamic "ingress" {
    for_each = [80, 443]
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Best used: when the number of nested blocks comes from data. Don't use it to hide a fixed, readable list; HashiCorp advises

against overusing it.

Exam traps

- **`count`, `for_each`, `depends_on`, and `provider` are meta-arguments, not blocks.** `lifecycle` is the only meta-argument written as a block.
- **`data` reads; `resource` manages.** Deleting a `data` block never deletes anything in AWS.
- **`check` warns; `precondition`/`postcondition` and `validation` fail.**
- **`sensitive = true` hides a value from output but still stores it in state.** `ephemeral` keeps it out of state entirely.
- **Block names vs reference names:** `locals` → `local.x`; `variable` → `var.x`; `data` → `data.type.name`; `module` → `module.name.output`.

- **Declarative blocks have CLI alternatives:** `moved` → `state mv`, `import` → `terraform import`, `removed` → `state rm`. The commands still exist.

See custom conditions for evaluation timing. See state operations for complete workflows.

Quick self-test

1. Which block looks up an existing VPC without managing it? → `data`
2. You renamed `aws_instance.web` to `aws_instance.app`. Which block stops a destroy/create? → `moved`
3. Which block declares the AWS provider version constraint? → `required_providers` inside `terraform`
4. A health assertion should warn but never block `apply`. Which block? → `check`
5. How do you reference a local named `prefix`? → `local.prefix`
6. Which block keeps a secret out of the state file entirely? → `ephemeral`

7. Stop managing a bucket but leave it in AWS? → `removed` with `lifecycle { destroy = false }`
8. Generate one `ingress` block per port in a list? → `dynamic "ingress"`
9. Is `for_each` a block? → No, it's a meta-argument

CHAPTER 07 / 18

Terraform Variables, Types, and Outputs

Associate 004 · Terraform 1.12 · Objectives 4c–4d | Study map

Follow the value

```
Caller input → var.environment →
local.label → resource → output
```

Local lab: Save this as `main.tf` in an empty directory.

```
variable "environment" {
  type        = string
  description = "Environment name."
  default     = "dev"
  nullable    = false
}

locals {
  label = "api-${var.environment}"
}
```

```
resource "terraform_data" "label" {
  input = local.label
}

output "label" {
  value = terraform_data.label.output
}
```

```
terraform init
terraform apply -
var='environment=test'
terraform output -raw label
```

Expected: `api-test`. Confirm the apply when Terraform asks.

Construct / Set by / Reference

> **Input variable**
>
> **Set by:** Caller, input file, environment, or default
>
> **Reference:** `var.environment`

> **Local value**
>
> **Set by:** An expression inside the module
>
> **Reference:** `local.label`

Root output

Set by: An expression inside the root module

Reference: `terraform output label`

Child output

Set by: An expression inside the child module

Reference: `module.child.label`

Reference: Values in modules.

Select a type

Type / Example value / Rule

`string`

Example value: `"dev"`

Rule: Text

`number`

Example value: 3

Rule: Numeric value

bool

Example value: `true`

Rule: True or false

list(string)

Example value: `["web", "api", "web"]`

Rule: Ordered. Duplicates allowed.

set(string)

Example value: `toset(["web", "api"])`

Rule: Unique members. No numeric index.

map(number)

Example value: `{ web = 2, api = 3 }`

Rule: String keys. One element type.

tuple([string, number])

Example value: `["web", 2]`

Rule: Fixed positions with specified types

`object({ name = string, replicas = number })`

Example value: `{ name = "web", replicas = 2 }`

Rule: Named attributes with specified types

Lists, sets, and maps are collection types. Tuples and objects are structural types. A literal such as `["web", 2]` starts as a tuple. Terraform can convert compatible values to satisfy a declared type.

Reference: Type constraints.

Use an object

Excerpt: Declare a group of related settings.

```
variable "service" {
  type = object({
    name     = string
    replicas = number
    tags     = optional(map(string), {})
  })
```

```
  default = {
    name     = "api"
    replicas = 2
  }
}

locals {
  service_name = var.service.name
  copies       = var.service.replicas
  tags         = var.service.tags
}
```

Expected: `service_name` is `"api"`. `copies` is `2`. `tags` is an empty map.

Distinguish null, unknown, and sensitive

Value / Meaning / Example

> **`null`**
>
> **Meaning:** No value
>
> **Example:** Omit an optional resource argument

Unknown

Meaning: Terraform will learn the value later

Example: An ID displayed as `(known after apply)`

Sensitive

Meaning: Terraform hides the value in normal output

Example: A marked password variable

`null` is not an empty string or an unknown value. An explicit `null` can override a variable default when `nullable = true`. With `nullable = false` and a default, a null input uses that default. Without a usable default, a non-nullable variable rejects null.

The type keyword `any` asks Terraform to infer a suitable type. It does not mean that every element of `list(any)` can retain a different type.

Inputs and outputs: exam details

- A variable without a default requires a value.
- A `.tfvars` file assigns input values. It does not declare variables.
- Terraform automatically loads variable files for the root module, not child modules.
- A child receives values through arguments in its `module` block.
- `terraform output` reads root outputs from state. It does not refresh infrastructure.
- `terraform output -json` returns machine-readable values.
- `-raw` is useful for a single string, number, or boolean output.
- Sensitive outputs still require protection. `-raw` and `-json` can reveal their values.

References: Input variables and output command. See input precedence for loading order.

Self-test

1. Which type removes duplicates and has no numeric index?
2. Does a child module automatically receive the root's variables?
3. Does `sensitive = true` remove a value from state?
4. Is `(known after apply)` the same as `null`?

Answers: 1. A set. 2. No. Pass arguments. 3. No. 4. No.

CHAPTER 08 / 18

Terraform Variable Precedence (Associate 004)

Scope: Local CLI input loading. | Study map
HCP Terraform adds workspace and variable-set precedence. See the HCP guide.

When Terraform finds a value for the same variable in several places, the one **loaded last wins**.

The order (lowest → highest priority)

```
1. default in the variable block
(variables.tf)        ← weakest
2. Environment variables
TF_VAR_<name>
3. terraform.tfvars
(auto-loaded)
4. terraform.tfvars.json
(auto-loaded)
5. *.auto.tfvars / *.auto.tfvars.json
```

```
(auto-loaded, alphabetical order)
6. -var and -var-file on the CLI
(in the order typed) ← strongest
```

If none of these sets a value and there's no `default`, Terraform **prompts** you interactively (or errors with `-input=false`).

Mnemonic, weakest to strongest: Don't **E**ver **T**rust **A** **C**ommand: **D**efault, **E**nv, **T**fvars, **A**uto.tfvars, **C**LI

Each source

/ Source / Example / Remember

1

Source: `default`

Example: `variable "region" { default = "us-east-1" }`

Remember: Fallback only. Any other source overrides it.

2

Source: Environment

Example: `export TF_VAR_region=us-west-2`

Remember: Prefix must be `TF_VAR_`; the rest matches the variable name. Common in CI and for secrets.

3

Source: `terraform.tfvars`

Example: `region = "us-west-2"`

Remember: Loaded automatically, but **only this exact name**.

4

Source: `terraform.tfvars.json`

Example: `{ "region": "us-west-2" }`

Remember: Same as #3 in JSON; loaded after it.

5

Source: `*.auto.tfvars`

Example: `prod.auto.tfvars`

Remember: Loaded automatically; several files load alphabetically, so the last one alphabetically wins.

6

Source: CLI

Example: `terraform apply -var="region=eu-west-1"` or `-var-file=prod.tfvars`

Remember: Highest priority. When you pass several, the **last one typed** wins.

Picking a var file on the command line

Keep one `.tfvars` file per environment and choose it at run time:

```
aws/
├── main.tf
```

```
├── variables.tf
├── testing.tfvars     #
instance_type = "t3.micro"
└── prod.tfvars        #
instance_type = "m5.large"
```

```
terraform plan    -var-
file="testing.tfvars"
terraform apply   -var-
file="testing.tfvars"
terraform destroy -var-
file="testing.tfvars"
```

- **Pass the same file to every command.** If `apply` doesn't get the same `-var-file` as `plan`, it uses different values. Alternatively, save the plan and apply that; it already contains the values:

  ```
  terraform plan -var-
  file="testing.tfvars" -out=tfplan
  terraform apply tfplan
  ```

  Ephemeral input values are not stored in the plan. Supply them again when required.

- **It overrides the auto-loaded files** (`terraform.tfvars`, `*.auto.tfvars`), which still load first.
- **You can stack files.** In `-var-file="common.tfvars" -var-file="prod.tfvars"`, values in `prod.tfvars` win for any key both files set.
- The path is relative to the directory you run Terraform in. The quotes are optional unless the path has spaces.

Exam traps

- **A file named `prod.tfvars` is not loaded automatically.** You must pass `-var-file=prod.tfvars`. Only `terraform.tfvars`, `terraform.tfvars.json`, and `*.auto.tfvars(.json)` are auto-loaded.
- **`-var-file` and `-var` have the same priority level.** Among them, order on the command line decides: in `-var-`

`file=a.tfvars -var="region=x"`, the `-var` wins because it comes later.

- **Environment variables are near the bottom.** They lose to every tfvars file, even though they feel "global".
- **`variables.tf` is just a name for a file.** It holds declarations; its `default` is the weakest value.

Quick self-test

1. `default = "a"`, `TF_VAR_x=b`, `terraform.tfvars` sets `"c"`. Result? → `c`
2. `terraform.tfvars` sets `"c"`, `z.auto.tfvars` sets `"d"`. Result? → `d`
3. `a.auto.tfvars` sets `"1"`, `b.auto.tfvars` sets `"2"`. Result? → `2`
4. `terraform.tfvars` sets `"c"`, CLI has `-var="x=e"`. Result? → `e`
5. Does `dev.tfvars` load without a flag? → No; use `-var-file=dev.tfvars`

6. No value anywhere and no default. What happens? → Terraform prompts for it
7. You ran `plan -var-file="testing.tfvars"`, then plain `terraform apply`. Which values does `apply` use? → Not `testing.tfvars`; it re-plans with only the auto-loaded files
8. `-var-file="common.tfvars" -var-file="prod.tfvars"`, both set `region`. Which wins? → `prod.tfvars` (last one typed)

Terraform Expressions and Functions

Associate 004 · Terraform 1.12 · Objective 4e | Study map

Read and evaluate expressions

Run `terraform console` in an empty lab directory. Enter one expression at a time. Use `exit` to stop the console. The examples below need no cloud credentials.

Console expression / Result

```
2 + 3
5
```

```
3 > 2 && true
true
```

```
true ? "large" : "small"
```
```
"large"
```

```
"api-${upper("dev")}"
```
```
"api-DEV"
```

```
[for n in ["web", "api"] : upper(n)]
```
```
["WEB", "API"]
```

```
[for n in [1, 2, 3] : n if n > 1]
```
```
[2, 3]
```

```
{for n in ["web", "api"] : n ⇒ length(n)}
```
```
{ api = 3, web = 3 }
```

```
[{id = "a"}, {id = "b"}][*].id
```
```
["a", "b"]
```

A `for` expression transforms values. It does not create resources. Both branches of a conditional must have compatible result types.

Reference: Expressions.

Select a function

Function example / Result or use

`length(["a", "b"])`

`2`

`contains(["dev", "prod"], "qa")`

`false`

`lookup({dev = 1}, "prod", 2)`

`2`

`merge({env = "dev"}, {env = "prod"})`

`{ env = "prod" }`

concat([1], [2, 3])

`[1, 2, 3]`

flatten([[1, 2], [3]])

`[1, 2, 3]`

toset(["a", "a", "b"])

A set with `"a"` and `"b"`

join("-", ["api", "dev"])

`"api-dev"`

split(",", "a,b")

`["a", "b"]`

coalesce(null, "", "dev")

`"dev"`

try({name = "api"}.port, 8080)

`8080`

`can(tonumber("bad"))`

`false`

`jsonencode({enabled = true})`

A JSON string

`jsondecode("{\"port\":8080}").port`

`8080`

`file("./policy.json")`

Read an existing file as text

`templatefile("./config.tftpl", {port = 8080})`

Render an existing template

`cidrsubnet("10.20.0.0/16", 8, 1)`

`"10.20.1.0/24"`

`try` catches dynamic evaluation errors. It does not repair an undeclared variable reference. `file` and `templatefile` need

files that exist before the run. Terraform does not use those functions to wait for a resource that creates a file.

References: Functions and try.

Create instances with count or for_each

Local lab: Save this as `main.tf` in a separate directory.

```
resource "terraform_data" "numbered"
{
  count = 2
  input = "worker-${count.index}"
}

resource "terraform_data" "named" {
  for_each = {
    web = 80
    api = 8080
  }
  input = "${each.key}:${each.value}"
}
```

```
terraform init
terraform plan
```

Expected: Four resource instances.

Expression / Address or value

First numbered instance

`terraform_data.numbered[0]`

API instance

`terraform_data.named["api"]`

`each.key` in the API instance

`"api"`

`each.value` in the API instance

`8080`

Use count when... / Use for_each when...

A numeric quantity identifies instances

Stable keys identify instances

An index is sufficient

Each instance has separate settings

Changing a list position can affect instances that use `count`. Changing a `for_each` key changes the instance address. The `for_each` value must be a map or a set of strings. Convert a list explicitly with `toset` when appropriate. For a set, `each.key` and `each.value` are the same string. Instance keys must be known before apply. Sensitive keys are not allowed. Do not use `count` and `for_each` together in one block.

Reference: for_each.

Generate nested blocks with dynamic

Excerpt: Generate two ingress blocks inside one security group.

```
resource "aws_security_group" "web" {
  name_prefix = "exam-web-"
  vpc_id      = aws_vpc.lab.id

  dynamic "ingress" {
    for_each = [80, 443]
    content {
      from_port   = ingress.value
```

```
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["10.20.0.0/16"]
    }
  }
}
```

This creates one resource with repeated nested settings. Unlike resource `for_each`, dynamic-block `for_each` can iterate a list. `dynamic` cannot generate `lifecycle` or `provisioner` blocks. It does not declare new top-level resource blocks.

Reference: Dynamic blocks.

Self-test

1. Does a `for` expression create resource instances?
2. Which map wins in `merge(a, b)` when both contain the same key?
3. Can a generated resource ID be an unknown `for_each` key during planning?
4. Does `dynamic` create several resources or repeated nested blocks?

Answers: 1. No. 2. b. 3. No. 4. Repeated nested blocks.

Terraform Dependencies and Lifecycle

Associate 004 · Terraform 1.12 · Objectives 4b, 4f | Study map

References create dependencies

Excerpt: The subnet needs the network ID.

```
resource "aws_vpc" "lab" {
  cidr_block = "10.20.0.0/16"
}

resource "aws_subnet" "app" {
  vpc_id     = aws_vpc.lab.id
  cidr_block = "10.20.1.0/24"
}
```

```
Create:   network → subnet
Destroy:  subnet → network
```

Terraform builds a directed acyclic graph (DAG) from dependencies. Independent resources can run in parallel. Circular references produce a dependency-cycle error. The order of files and blocks does not determine resource order.

Reference: Dependency graph.

Use explicit dependencies for hidden requirements

Excerpt: An instance needs an IAM policy attachment before it starts. Other instance arguments are omitted.

```
resource "aws_instance" "app" {
  ami                  = var.ami_id
  instance_type        = "t3.micro"
  iam_instance_profile =
aws_iam_instance_profile.app.name

  depends_on =
[aws_iam_role_policy.app]
}
```

The instance already references the instance profile. The separate policy is a behavioral dependency without a direct value reference.

- Prefer a normal reference when a resource needs another resource's value.
- Use `depends_on` when the required behavior is not represented by a reference.
- Use object references, not quoted addresses, in `depends_on`.
- A module-wide dependency can delay all resources and data reads inside the module.
- Extra dependencies can make plans more conservative and produce more unknown values.

Reference: depends_on.

Read resource addresses

Address / Meaning

`aws_vpc.lab.id`

Attribute of a managed resource

`data.aws_vpc.existing.id`

Attribute returned by a data source

`aws_instance.app[0].id`

Attribute of a `count` instance

`aws_instance.app["api"].id`

Attribute of a `for_each` instance

`module.network.vpc_id`

Child-module output

`module.network.aws_vpc.lab`

Resource address used by state commands

Normal configuration accesses child values through outputs. It cannot directly read a child's internal resource attributes. In shell commands, quote addresses that contain brackets.

```
terraform state show
'aws_instance.app["api"]'
```

Reference: Resource addressing.

Select a lifecycle rule

Rule / Effect / Limit

> **`create_before_destroy = true`**
>
> **Effect:** Create the replacement before deleting the old object
>
> **Limit:** Both objects must be able to exist together

`prevent_destroy = true`

Effect: Reject a plan that destroys or replaces the object

Limit: Removing the entire resource block removes this protection

`ignore_changes = [tags["Owner"]]`

Effect: Ignore that attribute when planning updates

Limit: The value is still used during creation

`replace_triggered_by = [terraform_data.release]`

Effect: Replace this resource when the referenced resource changes

Limit: Requires managed-resource references, not a plain variable

Lifecycle rules do not turn every update into a replacement. `create_before_destroy` changes the order when replacement is already required. Unique names, quotas, and

capacity can prevent both objects from existing together. The rule can also propagate to dependencies.

Reference: Lifecycle rules.

See replacement order

Local lab: Save this as `main.tf` in an empty directory.

```
variable "release" {
  type    = string
  default = "v1"
}

resource "terraform_data"
"deployment" {
  input            = var.release
  triggers_replace = var.release

  lifecycle {
    create_before_destroy = true
  }
}
```

```
terraform init
terraform apply
```

```
terraform plan -var='release=v2'
```

Expected: The second plan replaces the resource. `+/-` means create, then destroy. Without the lifecycle rule, replacement normally shows `-/+`: destroy, then create.

You can request replacement without editing a resource argument:

```
terraform plan -
replace=terraform_data.deployment
```

`-replace` does not bypass `prevent_destroy`. Use `-target` only for exceptional recovery or troubleshooting. A targeted plan can omit other required changes. Follow it with a complete plan.

Reference: Plan options.

Self-test

1. Does moving a block above another block change its dependency order?

2. Does `create_before_destroy` remove naming constraints?
3. Does deleting a resource block retain its `prevent_destroy` protection?
4. Should you add `depends_on` when a normal attribute reference already expresses the dependency?

Answers: 1. No. 2. No. 3. No. 4. Usually no.

Terraform Custom Conditions

Associate 004 · Terraform 1.12 · Objective 4g | Study map

Select the correct condition

Construct / Location / Use / False result

`validation`

Location: Inside `variable`

Use: Reject an invalid input

False result: Error

`precondition`

Location: Resource/data `lifecycle`, or directly inside `output`

Use: Check an assumption before the associated operation

False result: Error

`postcondition`

Location: Resource/data `lifecycle`

Use: Check a resulting value

False result: Error. Blocks dependent work.

`check` with `assert`

Location: Top level

Use: Report a health or configuration problem

False result: Warning. The run continues.

`terraform validate` checks configuration structure and internal consistency. It does not replace conditions that need actual input values or provider results. Conditions run when their required values are available. Unknown values can defer a condition until apply.

Reference: Custom validation.

Compare all four in one local lab

Save this complete example as `main.tf` in an empty directory.

```
variable "environment" {
  type    = string
  default = "dev"
}

variable "replicas" {
  type    = number
  default = 1

  validation {
    condition     = var.replicas ⩾ 1
&& floor(var.replicas) ⩵
var.replicas
    error_message = "Set replicas to
a positive integer."
  }
}

resource "terraform_data" "service" {
  input = {
    environment = var.environment
    replicas    = var.replicas
  }

  lifecycle {
    precondition {
      condition     = var.environment
```

```
≠ "prod" || var.replicas ≥ 2
      error_message = "Production
requires at least two replicas."
    }

    postcondition {
      condition     =
self.output.replicas ≥ 1
      error_message = "The result
must contain at least one replica."
    }
  }
}

output "service" {
  value =
terraform_data.service.output

  precondition {
    condition     =
length(var.environment) > 0
    error_message = "Set a non-empty
environment name."
  }
}

check "redundancy" {
  assert {
    condition     = var.replicas ≥ 2
    error_message = "This service has
no replica redundancy."
```

```
  }
}
```

```
terraform init
terraform validate
terraform plan
terraform plan -var='replicas=0'
terraform plan -
var='environment=prod'
terraform plan -
var='environment=prod' -
var='replicas=2'
```

Command / Expected

> **`validate`**
>
> Configuration is valid

> **Default `plan`**
>
> Warning from `check`. Planning continues.

> **`plan -var='replicas=0'`**
>
> Input-validation error

`plan -var='environment=prod'`

Precondition error because replicas defaults to 1

Production with 2 replicas

The input checks pass

Check assumptions and results

Need / Use

Permit only `dev` and `prod` input values

Variable validation with `contains`

Require an x86 image before creating a server

Resource precondition

Require an encrypted volume returned by a data source

Data-source postcondition

> **Report an unavailable application endpoint**
>
> A `check` block

Excerpt: Check a data-source result.

```
data "aws_ebs_volume" "existing" {
  filter {
    name   = "volume-id"
    values = [var.volume_id]
  }

  lifecycle {
    postcondition {
      condition     = self.encrypted
      error_message = "The selected
volume must be encrypted."
    }
  }
}
```

`self` refers to the current object in a postcondition. A failed postcondition does not roll back changes already made. An output supports `precondition`, but not `postcondition`.

Reference: Lifecycle condition rules.

Exam details

- A false `check` assertion warns. Invalid HCL can still make the configuration fail.
- A check runs during Terraform operations. It is not a continuous local monitoring service.
- HCP Terraform health assessments can run checks again between normal runs.
- `condition` is a boolean expression.
- `error_message` explains what the caller must correct.
- A condition that uses resource results can run later than one that uses known input values.

Reference: Check blocks.

Self-test

1. Which condition reports a problem without blocking a normal run?
2. Can an output contain a `postcondition`?

3. Does `validate` prove that a cloud API will accept every operation?
4. Does a failed postcondition undo an already-created resource?

Answers: 1. A `check` assertion. 2. No. 3. No. 4. No.

Terraform Sensitive Data and Ephemeral Values

Associate 004 · Terraform 1.12 · Objective 4h | Study map

Redaction and storage are separate

Mechanism / Purpose / Can the value remain in state or a saved plan?

> **`sensitive = true`**
>
> **Purpose:** Hide a value in normal CLI/UI output
>
> **Can the value remain in state or a saved plan?:** Yes

`ephemeral = true`

Purpose: Omit a variable or child output from state and plans

Can the value remain in state or a saved plan?: No, for that ephemeral value

`ephemeral` resource

Purpose: Obtain or generate temporary data

Can the value remain in state or a saved plan?: Terraform omits that resource's values

Write-only resource argument

Purpose: Send a value to a provider without storing that argument

Can the value remain in state or a saved plan?: Terraform omits that argument

Sensitive values can propagate through expressions. An output that exposes a sensitive value must be marked sensitive.

Redaction does not encrypt state. `terraform output -raw` and `-json` can reveal sensitive outputs.

Reference: Sensitive data.

Mark a temporary input

Excerpt: Declare a secret without including a secret value.

```
variable "db_password" {
  type      = string
  sensitive = true
  ephemeral = true
}
```

Supply the value through an approved secret source at run time. Both settings are useful: one controls display, and the other controls persistence. Marking a value ephemeral does not erase older state versions that already contain it.

Select a permitted destination

Destination for an ephemeral value / Allowed?

Destination for an ephemeral value:
Provider configuration

Allowed?: Yes

Destination for an ephemeral value: **A provider-supported write-only argument**

Allowed?: Yes

Destination for an ephemeral value:
Another ephemeral resource

Allowed?: Yes

Destination for an ephemeral value: **An ephemeral variable or child-module output**

Allowed?: Yes

Destination for an ephemeral value: A local expression used only in permitted ephemeral contexts

Allowed?: Yes

Destination for an ephemeral value: A provisioner or connection configuration

Allowed?: Yes

Destination for an ephemeral value: A normal stored resource argument

Allowed?: No

Destination for an ephemeral value: A root-module output

Allowed?: No

Destination for an ephemeral value: Resource instance keys that Terraform must persist

Allowed?: No

Ephemeral variables and resources require Terraform 1.10 or later. Write-only arguments require Terraform 1.11 or later. Both are available within the exam's Terraform 1.12 scope. The selected provider must also support the resource or argument.

Pass a value to a write-only argument

Excerpt: Other database arguments and provider setup are omitted.

```
resource "aws_db_instance" "app" {
  identifier = "exam-app"

  password_wo         =
var.db_password
  password_wo_version = 1
}
```

The provider receives `password_wo`. Terraform does not store that argument in state or the plan. The normal `password_wo_version` value is stored. Increase the version when you intend to

update the password. Terraform cannot compare old and new password values because it does not retain them.

A write-only argument can also accept a normal value. It does not remove copies of that value stored by another resource or output. The `_wo` suffix is a provider naming convention, not a suffix you can add to any argument.

Reference: Write-only arguments.

Generate temporary data

Excerpt: Requires a Random provider version that supports ephemeral resources.

```
ephemeral "random_password" "session"
{
  length  = 24
  special = false
}
```

Reference the result as `ephemeral.random_password.session.result`. Use a permitted destination, such as a

write-only argument. Terraform does not retain the generated value for the next run. If later access is required, store the secret in a suitable external secret service.

Reference: Ephemeral resources.

Understand Vault's role

```
Workload authentication → Vault →
temporary credential → provider → API
```

Vault can store secrets or issue credentials with a limited lifetime. Terraform uses the Vault provider to interact with Vault. Vault authorization policies determine what the caller can access.

Excerpt: A traditional data-source read still has a state risk.

```
provider "vault" {
  address =
"https://vault.example.com"
}

data "vault_kv_secret_v2" "app" {
  mount = "secret"
```

```
  name  = "exam/app"
}
```

The provider requires authentication from an approved external source. Fetching a value from Vault does not automatically make it ephemeral. A normal data source can place returned secrets in state. Short credential lifetimes do not remove the need to protect state.

Reference: Vault provider security.

Protect all copies

- Keep secret values out of configuration and version control.
- Keep state, saved plans, and diagnostic logs out of version control.
- Use access controls and encryption for remote state.
- Protect older state snapshots and backups.
- Prefer temporary workload credentials when the platform supports them.

- Treat `TF_VAR_` as input transport. It does not encrypt or automatically redact a value.

Self-test

1. Does `sensitive` keep a password out of state?
2. Can a root output be ephemeral?
3. Why does a write-only password often have a separate version argument?
4. Does a normal Vault data source guarantee that no secret enters state?

Answers: 1. No. 2. No. 3. To signal an update without comparing stored passwords. 4. No.

Terraform Modules

Associate 004 · Terraform 1.12 · Objectives 5a–5d | Study map

A module is a directory of configuration

Term / Meaning

Root module

The configuration directory where Terraform runs

Child module

A module called by another module

Input

A variable declared by the child and assigned by its caller

> **Output**
>
> A value that the child exposes to its caller

Terraform combines `.tf` files in one directory into one module. It does not automatically load `.tf` files from subdirectories. File names such as `main.tf` and `variables.tf` are conventions.

Reference: Modules.

Build and call a local module

Local lab: Create these two files in an empty lab directory.

```
lab/
├── main.tf
└── modules/
    └── label/
        └── main.tf
```

Root `main.tf`:

```
variable "environment" {
  type    = string
```

```
  default = "dev"
}

module "label" {
  source = "./modules/label"
  name   = var.environment
}

output "service_label" {
  value = module.label.label
}
```

Child `modules/label/main.tf`:

```
variable "name" {
  type = string
}

resource "terraform_data" "label" {
  input = "api-${var.name}"
}

output "label" {
  value = terraform_data.label.output
}
```

Run these commands from the root directory:

```
terraform init
terraform apply -
var='environment=test'
```

```
terraform output -raw service_label
terraform state list
```

Expected: Output `api-test`. State address `module.label.terraform_data.label`.

Follow variable scope

```
Root var.environment
       │ explicit name argument
       ▼
Child var.name → child resource →
child output label

│

▼

module.label.label
```

- The child does not automatically receive the root's variables or locals.
- A root `.tfvars` file supplies root inputs only.
- Pass values between sibling modules through their parent.

- Read child outputs, not child resource internals, in normal expressions.
- A missing required child input is a configuration error.

Reference: Module composition.

Select and version a source

Source / Example / Select a version with

Local directory

Example: `./modules/label`

Select a version with: The repository revision

Public registry

Example: `terraform-aws-modules/vpc/aws`

Select a version with: `version = "~> 5.0"`

Private registry

Example:
`app.terraform.io/example/network/aws`

Select a version with: A `version` constraint

Git repository

Example:
`git::https://example.com/team/network.git?ref=v1.2.0`

Select a version with: Git `ref`

Git subdirectory

Example:
`git::https://example.com/team/modules.git//network?ref=v1.2.0`

Select a version with: Git `ref` after the subdirectory

> **Supported archive/object storage source**
>
> **Example:** A module archive URL
>
> **Select a version with:** The selected archive location

`version` is for registry modules. Do not add it to local or Git module calls. Use a literal source address in Terraform 1.12. Run `terraform init` after adding or changing a module source. Use `terraform init -upgrade` to update eligible installed modules and providers. The dependency lock file records providers, not remote module versions. Use an exact registry version or Git commit when you need a fixed module revision.

Reference: Module sources.

Pass provider configurations

Excerpt: The child uses its default `aws` name. The root supplies its `west` configuration.

```
provider "aws" {
  alias  = "west"
  region = "us-west-2"
}

module "network" {
  source = "./modules/network"

  providers = {
    aws = aws.west
  }
}
```

Default provider configurations can pass from parent to child automatically. Aliased configurations need explicit mapping. Each module declares its own provider requirements and compatible versions.

If a child refers to an alias such as `aws.replica`, declare it in the child's requirements:

```
terraform {
  required_providers {
    aws = {
      source                =
"hashicorp/aws"
      configuration_aliases =
[aws.replica]
    }
  }
}
```

The parent then maps `aws.replica` to an actual configured provider. Keep provider connection blocks in the root for reusable modules. Legacy children with their own provider blocks cannot use module `count`, `for_each`, or `depends_on`.

Reference: Providers within modules.

Self-test

1. Does Terraform load a child directory without a `module` call?
2. Can a child automatically read `var.environment` from its parent?
3. Can `version` select a Git tag?

4. Do aliased providers pass to a child automatically?

Answers: 1. No. 2. No. 3. No. Use `ref`. 4. No.

CHAPTER 14 / 18

Terraform State and Backends

Associate 004 · Terraform 1.12 · Objectives 2d, 6a–6c | Study map

Know what state records

```
Configuration address       State
binding        Real object
aws_vpc.lab                 id = vpc-
example      AWS network
```

State connects resource addresses to real objects. It also records resource attributes, dependency metadata, and provider associations. Cached attributes help Terraform work with large configurations. State is not the desired configuration and is not a live inventory of every cloud resource.

Artifact / Purpose

`.tf` files

Describe the desired configuration

Resource state

Record Terraform's known objects and their attributes

Saved plan

Record proposed actions for a later apply

`.terraform.lock.hcl`

Select provider versions and verify packages

`.terraform/`

Store local initialization data and downloaded dependencies

Deleting state does not delete the real resources. Terraform can lose their bindings and propose duplicate creation.

Reference: Purpose of state.

Select a storage location

Storage / Resource state location / Execution location

Default local backend

Resource state location: `terraform.tfstate` in the working directory

Execution location: Local machine

S3, Azure Storage, or GCS backend

Resource state location: Remote storage

Execution location: Usually the machine running the CLI

HCP Terraform with remote execution

Resource state location: HCP workspace

Execution location: HCP workers

HCP Terraform with local execution

Resource state location: HCP workspace

Execution location: Local machine

Remote state storage does not automatically mean remote execution. Backend types are built into Terraform. They are not installed as provider plugins.

Excerpt: Store local state at a different path.

```
terraform {
  backend "local" {
    path = "state/lab.tfstate"
  }
}
```

The local backend supports locking on the local system. Local files are inconvenient for multiple people on different machines.

Reference: Local backend.

Configure a remote backend

Excerpt: This requires an existing S3 bucket and suitable AWS permissions.

```
terraform {
  backend "s3" {
    bucket       = "example-team-
tfstate"
    key          =
"exam/dev/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

- Create the storage before initialization. This backend block does not create its bucket.
- Use one backend block per configuration.
- Do not use `var`, `local`, or resource references inside backend settings.
- Do not combine `backend` and `cloud` in the same configuration.
- Supply backend credentials through its supported environment or credential

mechanism.

- Provider connection settings do not configure backend authentication.
- Backend settings can enter initialization files and saved plans. Keep secrets out of them.

References: Backend configuration and S3 backend.

Supply partial configuration

Excerpt: Keep the backend type in `main.tf`.

```
terraform {
  backend "s3" {}
}
```

Put non-secret settings in `state.s3.tfbackend`:

```
bucket       = "example-team-tfstate"
key          =
"exam/dev/terraform.tfstate"
region       = "us-east-1"
```

```
encrypt      = true
use_lockfile = true
```

```
terraform init -backend-
config=state.s3.tfbackend
```

The settings file contains arguments, not a second `terraform` block.

Migrate or reconfigure

Command / Effect

> **`terraform init -migrate-state`**
>
> Reinitialize and copy existing state to the new backend after confirmation

> **`terraform init -reconfigure`**
>
> Accept the current backend settings without migrating existing state

For a migration, stop other writers first. Keep a protected backup of the current state. Change the backend settings. Run initialization with `-migrate-state`. Then

inspect state and run a plan to check the bindings. `-reconfigure` is not a state-copy command. HCP `cloud` integration has its own initialization migration flow. See the HCP guide.

Reference: Initialization.

Understand state locking

```
Run A acquires the state lock → writes state → releases the lock
Run B cannot write the same state while Run A holds that lock
```

Locking prevents competing writers from corrupting state. Support depends on the backend and its configuration. Encryption, version history, and locking solve different problems.

```
terraform plan -lock-timeout=60s
terraform force-unlock LOCK_ID
```

The second command is for a stale lock that you own. First confirm that the original run has stopped. It does not stop another process or repair damaged state. Avoid `-lock=false` for ordinary team work.

Reference: State locking.

Recognize CLI workspaces

```
terraform workspace list
terraform workspace new dev
terraform workspace select default
terraform workspace show
```

A CLI workspace selects a separate state for the same working configuration. Creating one does not copy infrastructure or the current workspace's state. With local storage, non-default states are under `terraform.tfstate.d/<workspace>/`. Use `terraform.workspace` to read the selected workspace name in configuration. CLI workspaces do not provide separate credentials or access-control boundaries by themselves.

Reference: CLI workspaces.

Self-test

1. Does an S3 backend make AWS execute Terraform for you?
2. Does the dependency lock file prevent concurrent state writes?
3. Which initialization flag copies state to a new backend?
4. Can backend settings use `var.bucket_name`?

Answers: 1. No. 2. No. 3. `-migrate-state`. 4. No.

Terraform State Operations and Imports

Associate 004 · Terraform 1.12 · Objectives 6d, 7a–7b | Study map

Inspect before you change

Command / Shows

`terraform state list`

Resource and data-source addresses in state

`terraform state show 'aws_instance.app["api"]'`

Attributes for one state object

`terraform show`

A readable view of the latest state

`terraform show tfplan`

A saved plan

`terraform show -json tfplan`

A machine-readable plan

`terraform output`

Root-module outputs

`terraform state pull`

The raw current state snapshot

State inspection does not refresh the remote objects. State and JSON output can expose sensitive data. Use Terraform commands for changes. Do not edit state JSON by hand.

Reference: State commands.

Decide what to do with drift

Drift is a change to a managed object outside Terraform.

```
Configuration: instance size = small
Stored state:  instance size = small
Real object:   instance size = large
← someone changed it
```

Operation / Result

Normal `plan`

Read the real object and propose changes to match configuration

Normal `apply`

Reconcile the object with configuration

`plan -refresh-only`

Preview updates to state and root outputs

`apply -refresh-only`

Save refreshed state and outputs without changing infrastructure

Refresh-only does not edit `.tf` files. To keep an intentional external change, update configuration to match it too. Otherwise, a later normal apply can reverse that change.

```
terraform plan -refresh-only -
out=refresh.tfplan
terraform show refresh.tfplan
terraform apply refresh.tfplan
```

The old `terraform refresh` command writes refreshed state without a separate approval step. Prefer the refresh-only plan/apply workflow. `-refresh=false` can hide external changes. It is not a normal drift-management solution.

Reference: Refresh-only operations.

Rename a resource without replacement

Local lab, step 1: Save this as `main.tf` in an empty directory.

```
resource "terraform_data" "old" {
  input = "keep-this-object"
}
```

```
terraform init
terraform apply
```

```
terraform state show
terraform_data.old
```

Step 2: Replace the file contents with this configuration.

```
resource "terraform_data" "new" {
  input = "keep-this-object"
}

moved {
  from = terraform_data.old
  to   = terraform_data.new
}
```

```
terraform plan
terraform apply
terraform state list
```

Expected: The address changes to `terraform_data.new`. Terraform retains the object ID. There are no resource additions or destructions. A move can also change a resource's module address. Keep useful `moved` history so older module users can upgrade safely.

The command alternative changes state immediately:

```
terraform state mv terraform_data.old
terraform_data.new
```

Do not run the alternative after you already applied the move. Reference: Refactoring.

Stop management without destruction

Excerpt: Remove the corresponding resource block. Add this block instead.

```
removed {
  from = aws_s3_bucket.legacy

  lifecycle {
    destroy = false
  }
}
```

Review and apply the plan. Terraform removes the state binding and leaves the bucket intact. The command alternative is `terraform state rm`

`aws_s3_bucket.legacy`. If its resource configuration remains, the next plan proposes creating a replacement object. Simply deleting a resource block normally proposes destruction.

Reference: Remove resources from state.

Import an existing object

Import creates a state binding for an object that already exists. It does not duplicate the object. Use one Terraform resource address for each imported object.

Excerpt: The named bucket must already exist. Configure the provider first.

```
resource "aws_s3_bucket" "legacy" {
  bucket = "example-existing-bucket"
}

import {
  to = aws_s3_bucket.legacy
  id = "example-existing-bucket"
}
```

```
terraform init
terraform plan -out=import.tfplan
terraform show import.tfplan
terraform apply import.tfplan
```

The plan can contain both import actions and changes needed to match your configuration. Check every proposed action before applying. Import IDs and import support depend on the provider resource type.

Reference: Configuration-driven import.

Compare the two import methods

Method / Configuration required / When state changes

> **`import` block**
>
> **Configuration required:** Resource configuration, or supported generated configuration
>
> **When state changes:** On apply

> **`terraform import ADDRESS ID`**
>
> **Configuration required:** Write the resource configuration first
>
> **When state changes:** During the import command

Alternative: Use the CLI after initialization and after writing the resource block.

```
terraform import aws_s3_bucket.legacy
example-existing-bucket
terraform plan
```

The CLI command does not generate a resource block. For supported configuration generation, start with an `import` block and no matching resource block:

```
terraform plan -generate-config-
out=generated.tf
```

Use a new output path. Review the generated code, then plan again before applying. The `moved`, `removed`, and `import`

blocks are declarative alternatives. Their CLI counterparts still exist.

Reference: CLI import.

Self-test

1. Does refresh-only update the desired configuration?
2. Which block records a rename without destroying the object?
3. Which `removed` setting leaves the real object intact?
4. Does CLI import write a complete resource configuration?

Answers: 1. No. 2. `moved`. 3. `destroy = false`. 4. No.

Terraform Debugging

Associate 004 · Terraform 1.12 · Objective 7c | Study map

Match the failure to the tool

Symptom / First check

Invalid syntax or argument

Run `terraform validate` after initialization

Inconsistent style

Run `terraform fmt -check`

Missing plugin or module

Run `terraform init`

Conflicting provider versions

Inspect requirements with `terraform providers`

An unexpected resource change

Read `terraform plan` and the relevant state object

API authentication failure

Check the provider's credential source and permissions

State lock error

Check whether another run is active

Provider crash or unclear API behavior

Enable diagnostic logging

Verbose logs add detail. They do not correct the configuration or credentials.

Enable logs for one command

Run this in an initialized lab directory:

```
TF_LOG=DEBUG TF_LOG_PATH=./terraform-debug.log terraform plan
```

These environment assignments apply only to that command. `TF_LOG_PATH` selects a file. The file is appended to across runs. `TF_LOG_PATH` alone does not enable logging.

Level / Detail

`ERROR`

Errors

`WARN`

Warnings and errors

`INFO`

General progress and higher-severity messages

`DEBUG`

Diagnostic detail

`TRACE`

Most detail

`TF_LOG=JSON` requests JSON-formatted logs at TRACE detail. Treat the diagnostic format as unstable. Do not make application logic depend on it.

Reference: Terraform logging.

Enable logs for several commands

```
export TF_LOG=TRACE
export TF_LOG_PATH=./terraform-
trace.log
terraform plan
unset TF_LOG
unset TF_LOG_PATH
```

Without `TF_LOG_PATH`, Terraform writes diagnostic logs to standard error. Keep logs out of version control. They can contain secret values and request details.

Separate Core and provider detail

Alternative: Select independent log levels.

```
unset TF_LOG
TF_LOG_CORE=ERROR
TF_LOG_PROVIDER=TRACE terraform plan
```

`TF_LOG_CORE` controls Terraform Core logs. `TF_LOG_PROVIDER` controls provider logs. Use these when the failure is easier to isolate by component.

Read a failure in order

1. Read the error summary.
2. Find the resource address and source line.
3. Check whether failure occurred during initialization, validation, planning, or apply.
4. Check the related variable, provider, backend, or module.
5. Enable logs if the normal error lacks sufficient detail.
6. Correct the cause.
7. Run a new plan.

After a partial apply, some resources can already exist. Terraform records completed changes when possible. A new plan identifies remaining work. Repeating apply is not an automatic rollback.

Reference: Troubleshooting workflow.

Self-test

1. Which log level provides the most detail?
2. Does `TF_LOG_PATH` alone enable logs?
3. Do logs go to standard output or standard error by default?
4. Should a diagnostic log be committed with the configuration?

Answers: 1. `TRACE`. 2. No. 3. Standard error. 4. No.

CHAPTER 17 / 18

HCP Terraform

Associate 004 · Objectives 8a–8d · Reviewed 2026-10-14 | Study map

Know the service's purpose

HCP Terraform provides shared state, managed runs, and team controls. Terraform Enterprise provides a self-hosted platform with related capabilities. For this exam, focus on the HCP workflows and features in the objective list.

Need / HCP capability

Share state	Workspace state storage and version history
Run Terraform consistently	Managed execution and workspace settings

Review changes

Plans, approvals, and run history

Restrict access

Teams and permissions

Enforce organization rules

Policy checks and governance features

Reference: HCP Terraform overview.

Separate organization, project, and workspace

```
Organization: example-team
  ├── Project: network
  │     ├── Workspace: network-dev
  │     └── Workspace: network-prod
  └── Project: application
        ├── Workspace: app-dev
        └── Workspace: app-prod
```

Object / Contains or controls
Organization Shared administration, teams, and settings
Project A group of workspaces with related ownership and access
Workspace A configuration context, state, variables, runs, and settings

Putting two workspaces in a project does not merge their state. Use separate workspaces for infrastructure with separate ownership or change schedules.

Reference: Projects.

HCP and CLI workspaces are different

CLI workspace / HCP workspace

Selects a separate state for one working configuration

Has its own configuration context, state, variables, and run history

Uses the local CLI's available credentials

Uses credentials available to its execution environment

Does not provide team authorization by itself

Supports team access controls

Does not create a run service

Can execute and coordinate runs

Reference: Workspace comparison.

Separate workflow from execution mode

Workflow / What starts a run?

CLI-driven

A user runs `terraform plan` or `terraform apply`

VCS-driven

A connected version-control event starts the appropriate run

API-driven

Automation uploads configuration and creates a run through the API

Execution mode / Where Terraform executes

Execution mode: **Remote**

Where Terraform executes: HCP-managed workers

> **Execution mode:** **Agent**
>
> **Where Terraform executes:** An agent running in a network you control

> **Execution mode:** **Local**
>
> **Where Terraform executes:** The local machine. HCP stores state.

CLI-driven does not mean locally executed. An agent can reach private infrastructure while HCP coordinates the run. Local execution does not receive the same managed-run policy and execution features. HCP does not evaluate workspace variables or variable sets in local execution mode. Provide variables and credentials in the environment where Terraform actually executes.

References: Run workflows and execution settings.

Connect a local configuration

Excerpt: Use an existing organization and an appropriate workspace.

```
terraform {
  cloud {
    organization = "example-team"

    workspaces {
      name = "exam-lab"
    }
  }
}
```

```
terraform login
terraform init
terraform plan
terraform apply
```

`login` authenticates the CLI to HCP Terraform. It does not authenticate the AWS provider. The `cloud` block integrates with HCP state and operations. It cannot coexist with a `backend` block.

To migrate existing local state:

1. Stop other runs and keep a protected state backup.
2. Configure the `cloud` block for the intended workspace.
3. Run `terraform init`.
4. Review and confirm the offered state migration.
5. Check the destination state and run a plan.

This initialization flow differs from ordinary backend migration flags. Use the workspace settings to select its Terraform execution version. Keep that version compatible with `required_version` in configuration.

Reference: CLI integration.

Understand a managed run

```
Configuration received
        ↓
Initialization and plan
        ↓
Configured cost, policy, and external
checks
```

```
        ↓
Approval when required
        ↓
Apply and save state
```

A speculative plan previews changes and cannot be applied directly. Pull requests commonly produce speculative plans. A normal run can proceed to apply after required checks and approval. The configured workflow and auto-apply settings control approval behavior. HCP coordinates workspace runs to avoid competing state changes.

Reference: Remote operations.

Set variables where the run can use them

Variable kind / Example / Consumer

> **Terraform variable**
>
> **Example:** `region = "us-east-1"`
>
> **Consumer:** `var.region` in the configuration

Environment variable

Example: `AWS_REGION`

Consumer: Provider or process environment

Variable set

Example: Common team settings

Consumer: Selected workspaces or projects

Sensitive variable

Example: A secret supplied through the UI/API

Consumer: Run environment, with restricted display

For a complex Terraform value, use the HCL option when the value must be parsed as an expression. Shared variable sets reduce repeated configuration. Priority variable sets can override values at a more specific scope. Do not apply the local CLI precedence list to every HCP variable collision. Use the

documented HCP precedence rules for the relevant scopes. Marking a workspace input sensitive does not guarantee its absence from Terraform state.

Reference: HCP variable rules.

Connect workspaces with run triggers

```
network-prod applies successfully
               ↓ run trigger
app-prod queues a new run
```

The downstream workspace declares the upstream source workspace. A trigger starts work. It does not copy output values or grant state access. Configure data access separately when the downstream configuration needs upstream outputs. Trigger-created runs do not auto-apply unless the separate run-trigger auto-apply setting permits it.

`terraform_remote_state` exposes root outputs but requires access to the state snapshot. For HCP outputs, `tfe_outputs` can avoid granting full state access.

Reference: Run triggers and output sharing.

Select a collaboration or governance feature

Scenario / Feature	
Share approved modules and providers	Private registry
Let a team review plans without applying	Team permissions
Apply the same rules across workspaces	Policy sets

Check policy as code

Sentinel or Open Policy Agent (OPA)

Check a plan with an external service

Run tasks

Detect changes made outside Terraform

Drift detection through health assessments

Recheck infrastructure assertions

Continuous validation through health assessments

Find resources, providers, or versions across workspaces

Explorer

Offer controlled requests for infrastructure changes

Change requests

> **Avoid long-lived provider credentials**
>
> Dynamic provider credentials

Policy failure behavior depends on enforcement settings. A mandatory rule can block apply. An advisory rule can report a failure without blocking it. Do not assume all failing policies can be overridden. Health assessments detect problems. They do not automatically repair drift.

References: Policies, health, Explorer, and change requests.

Recognize dynamic provider credentials

```
HCP workload identity → cloud trust
policy → short-lived cloud
credentials
```

The cloud verifies the run's identity before it grants access. Trust settings can restrict the organization, workspace, and run phase. An

HCP login token and a cloud-provider credential serve different purposes.

Reference: Dynamic credentials.

Self-test

1. Does a project combine its workspaces into one state file?
2. Must a CLI-driven HCP run execute on the user's laptop?
3. Does a run trigger automatically grant access to upstream outputs?
4. Can a speculative plan be applied directly?
5. Which feature groups workspaces by ownership?
6. Does drift detection automatically restore the declared configuration?

Answers: 1. No. 2. No. 3. No. 4. No. 5. Projects. 6. No.

Terraform Tags on AWS

Tags are key/value labels attached to AWS resources. They answer **who owns this, what is it for, and what does it cost**.

Why tag

Use / Example

Cost tracking

Filter the AWS bill by `Project` or `CostCenter`

Ownership

Find who to ask before deleting something (`Owner`)

Environment separation

Tell `dev` from `prod` at a glance

Automation

Scripts that stop every instance tagged `Schedule = office-hours`

Access control (ABAC)

IAM policies that allow actions only on resources with a matching tag

Identification

`ManagedBy = terraform` marks what not to edit by hand

Where you can apply tags

There are three places, from broadest to most specific.

1. Provider `default_tags`: every resource from this provider

```
provider "aws" {
  region = "us-east-1"

  default_tags {
```

```
    tags = {
      Environment = "dev"
      Project     = "wikinotes-lab"
      Owner       = "chaz"
      ManagedBy   = "terraform"
    }
  }
}
```

- Applied automatically to **every taggable resource** this provider creates.
- Each provider block has its own `default_tags`, so a second provider with an `alias` (for example, another region) needs its own block too.
- **Use for:** tags that must be on everything in the configuration.

2. `locals` + `merge()`: a shared set, reused selectively

```
locals {
  common_tags = {
    Project   = "wikinotes-lab"
    ManagedBy = "terraform"
  }
}
```

```
resource "aws_s3_bucket" "logs" {
  bucket = "wikinotes-lab-logs"
  tags   = merge(local.common_tags, {
Name = "logs", DataClass = "internal"
})
}
```

- `merge()` combines maps; when the same key appears twice, the **later map wins**.
- This was the standard pattern before `default_tags` existed (AWS provider 3.38). You'll still see it in modules and older code.
- **Use for:** a group of tags shared by some resources but not all.

3. Resource `tags`: one resource only

```
resource "aws_instance" "web" {
  ami           = "ami-
0123456789abcdef0"
  instance_type = "t3.micro"

  tags = {
    Name = "web-01"
    Role = "frontend"
  }
}
```

- **Use for:** tags that are unique to a resource, especially `Name` (the label AWS shows in the console).

How the layers combine

```
default_tags  (provider)
Environment=dev   Owner=chaz
      +
tags          (resource)
Name=web-01       Environment=staging
      =
tags_all      (what AWS gets)
Name=web-01  Owner=chaz
Environment=staging
```

- **The resource wins** when both set the same key.
- `tags_all` is a read-only attribute that shows the final merged set. It's what you see in `plan` output, and you can reference it: `aws_instance.web.tags_all`.

Passing tags into modules

Give the module a `tags` variable and merge it with the module's own tags:

```
# modules/network/variables.tf
variable "tags" {
  type    = map(string)
  default = {}
}

# modules/network/main.tf
resource "aws_vpc" "this" {
  cidr_block = "10.0.0.0/16"
  tags       = merge(var.tags, { Name
= "main-vpc" })
}

# root main.tf
module "network" {
  source = "./modules/network"
  tags   = { CostCenter = "1234" }
}
```

The module's resources still get the provider's `default_tags`, because a module uses the provider passed down from the root.

Where tags *don't* reach automatically

`default_tags` only covers resources Terraform creates directly. Resources AWS creates for you need their own tag settings:

Case / What to do

EC2 instances launched by an Auto Scaling group

`tag { key, value, propagate_at_launch = true }` blocks on `aws_autoscaling_group` (it does not receive `default_tags`)

Instances and volumes from a launch template

`tag_specifications { resource_type = "instance" tags = { … } }` (repeat for `"volume"`)

EBS volumes created by `aws_instance`

`root_block_device { tags = { … } }` or `volume_tags`

Check `plan` and the AWS console after the first `apply` to confirm tags landed where you expect.

Best practices

1. **Define a standard tag set** and use it everywhere. A typical minimum: `Environment`, `Project`, `Owner`, `CostCenter`, `ManagedBy`.
2. **Put the standard set in `default_tags`.** No resource can forget it.
3. **Pick one casing style and keep it.** AWS tag keys are case-sensitive: `Environment` and `environment` are different tags.
4. **Use resource `tags` for `Name`** and anything else unique to that resource.
5. **Drive values from variables** (`Environment = var.environment`) so the same code tags dev and prod correctly.

6. **Enforce required tags** with variable validation:

```
variable "tags" {
  type = map(string)
  validation {
    condition     = alltrue([for k
in ["Owner", "Project"] :
contains(keys(var.tags), k)])
    error_message = "tags must
include Owner and Project."
  }
}
```

   Across a whole AWS organization, also use **AWS Organizations tag policies**.

7. **Activate cost allocation tags** in AWS Billing; otherwise tags don't show up in Cost Explorer.

8. **Never put secrets or personal data in tags.** Anyone with read access to the resource, the billing data, or the Terraform state can see them.

9. **If an outside tool adds tags**, stop Terraform from removing them on the next apply:

```
lifecycle {
  ignore_changes =
[tags["LastScannedBy"]]
}
```

AWS limits

- Up to **50 tags** per resource.
- Key up to **128** characters, value up to **256**.
- The `aws:` prefix is reserved for AWS and can't be used.

Quick self-test

1. Which block puts tags on every resource from a provider? → `default_tags` inside `provider "aws"`
2. Provider default sets `Environment=dev`, resource sets `Environment=prod`. Result? → `prod` (resource wins)
3. Which attribute shows the final merged tags? → `tags_all`

4. In `merge(a, b)`, which wins when both have the same key? → `b` (the later map)
5. Do ASG-launched instances get `default_tags`? → No; use `tag` blocks with `propagate_at_launch = true`
6. A second `provider "aws"` with `alias = "west"`. Does it inherit the first one's `default_tags`? → No; give it its own block
7. Why do tags not show in Cost Explorer? → Cost allocation tags weren't activated in Billing